
Measuring Doctest Executability in Python Corpora: Feasibility of Execution-Filtered SFT Data Curation

Anonymous Authors¹

Abstract

Only 1 in 1,000 Python files in a curated code corpus contains an independently-executable doctest — a $31\times$ gap below the 3.1% rate at which `>>>` doctest patterns appear. This finding emerges from a systematic three-phase feasibility scan of 10,000 randomly sampled Python files: Phase A (pattern detection: 3.1%), Phase B (AST extraction: 2.0%), and Phase C (subprocess execution: 0.1%). The collapse at Phase C is driven by import isolation: Python library code depends on third-party packages unavailable in clean subprocess environments. The resulting executable-doctest token pool (0.004M tokens estimated) is approximately $125,000\times$ below a 500M-token supervised fine-tuning (SFT) budget target, making doctest-passing filtering structurally infeasible for raw Python corpus SFT construction at this scale. We validate a three-phase filtering pipeline (28/28 unit tests passing, 129.8 seconds for 10,000 files) and identify `compile()`-based filtering as the practical execution-quality alternative — scalable to 12.96M+ files without dependency management. The equal-token-budget SFT comparison experiment (`compile`-only vs. unfiltered, Qwen2.5-Coder-1.5B, HumanEval/MBPP pass@1) is designed and designated as future work.

1. Introduction

We set out to filter Python training data for code large language models (LLMs) using doctest execution — a seemingly natural signal of functional correctness. Before training a single model, we discovered that only 1

in 1,000 Python files in a curated corpus contains an independently-executable doctest. The `>>>` patterns that Python conventions encourage appear in 3.1% of files, but $31\times$ fewer survive subprocess execution in a clean environment. This gap is not a measurement artifact: it is a structural property of Python library code, which depends on third-party packages unavailable in clean execution environments.

1.0.1. 1.1 MOTIVATION

Code LLMs are supervised fine-tuned on corpora of Python code drawn from platforms such as GitHub, producing models evaluated on benchmarks like HumanEval (?) and MBPP (?). These corpora are large but noisy: raw code repositories contain syntactically invalid files, functionally broken examples, and copy-paste debris. Training on such data exposes models to malformed patterns, potentially limiting benchmark performance relative to training on a cleaner, execution-validated subset.

The field has responded with heuristic filtering — removing files based on line length, alphanumeric fraction, and encoding (as in The Stack (?) and StarCoder [Li et al., 2023a]). A more principled alternative is execution-based filtering: retain only files that pass a syntax validity gate (`compile()`) or a functional correctness gate (doctest execution). The appeal of execution-based filtering is that it applies a ground-truth check — code that does not compile is certainly wrong, and code whose doctests fail is likely inconsistent with intended behavior.

However, execution-based filtering rests on an untested assumption: that the corpus contains sufficient executable examples to meet SFT token budget requirements. For the doctest-passing condition — the strictest functional gate — this assumption is precisely what we examine.

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

1.0.2. 1.2 THE DEEPER PROBLEM: PATTERN $\neq$ EXECUTABILITY

Python’s doctest convention was designed for installed environments. A docstring beginning with `>>> import numpy as np` is documentation intent, not a self-contained executable example: it requires `numpy` to be installed in the execution environment. Clean subprocess execution — without pre-installed third-party packages — will reject such examples at the import stage.

Prior work did not measure this gap. The Stack paper estimated `compile()` validity rates on 10,000 sampled Python files (?) but did not characterize doctest executability. `phi-1` (?) demonstrated that quality-filtered SFT data outperforms equal-token-budget unfiltered data, but used GPT-4 curation rather than execution gates. `EffiCoder` (?) applied execution-selected SFT samples for instruction tuning and achieved +13pp HumanEval improvement on Qwen2.5-Coder-7B (44.8% to 57.7%), but operated on instruction-response pairs rather than raw corpus code. No prior work performed a controlled comparison of unfiltered vs. `compile`-only vs. `doctest`-passing SFT filtering on a raw Python corpus at equal token budget.

1.0.3. 1.3 KEY INSIGHT

Execution filtering for code LLM data quality operates in two fundamentally different regimes: (1) `compile`-only (syntax gate), which retains a substantial fraction of files and scales to any corpus size; and (2) `doctest`-passing (functional gate), which collapses to approximately 0.1% of files in library-heavy Python corpora — a rate insufficient for SFT at 500M-token budgets. These are not points on a quality spectrum; they are categorically different strategies with different corpus-scale feasibility profiles.

1.0.4. 1.4 CONTRIBUTIONS

1. **First systematic measurement of the gap between `>>>` pattern prevalence and subprocess executability in a curated Python code corpus used for LLM training.** We conduct a three-phase pilot scan of 10,000 randomly sampled Python files, measuring the prevalence of `>>>` patterns (Phase A: 3.1%), AST-parseable doctest examples (Phase B: 2.0%), and independently-executable doctests (Phase C: 0.1%). The $31\times$ gap from Phase A to Phase C is the primary finding, with import errors as the dominant failure mode.

1. Validated multi-phase doctest filtering

pipeline.

We implement and release the Phase A/B/C pipeline — pattern check, AST extraction, and `base64`-isolated subprocess execution — which processes 10,000 files in 129.8 seconds with 4-worker parallelism and passes 28/28 unit tests.

1. Practical recommendation for SFT corpus construction.

We establish that `compile`-only filtering is the practical execution-quality gate for raw Python SFT corpus construction at scale, and that `doctest`-passing filtering requires dependency-aware execution infrastructure to be viable. The equal-token-budget SFT comparison experiment is designed and pending execution.

2. Related Work

2.0.1. 2.1 CODE LLM BENCHMARKS

HumanEval (?) established the `pass@k` metric for code generation evaluation, using 164 hand-written Python problems with unit tests. MBPP (?) provided 974 Python programming tasks ranging from simple to challenging. Both benchmarks evaluate functional correctness via execution.

2.0.2. 2.2 HEURISTIC-ONLY CORPUS FILTERING

StarCoder [Li et al., 2023a] and StarCoder 2 (?) established the standard pipeline for open-source code corpus construction from The Stack [Kocetkov et al., 2022]: near-deduplication, PII redaction, and heuristic quality filtering (line length, alphanumeric fraction, encoding, XML/HTML ratio). These filters remove obvious junk but do not apply execution-based quality signals. StarCoder achieves 40% HumanEval `pass@1` at 15.5B parameters.

Heuristic filtering cannot distinguish syntactically valid from invalid code or functionally correct from incorrect code.

2.0.3. 2.3 QUALITY CURATION VIA LANGUAGE MODELS

`phi-1` (?) demonstrated that 7B quality tokens curated by GPT-4 yield 50.6% HumanEval `pass@1` at 1.3B parameters — establishing that quality can outweigh quantity at equal token budget. `phi-1.5` [Li et al., 2023b] extended this to reasoning tasks. GPT-4-based quality curation, however, is not reproducible or scalable for open-source use.

2.0.4. 2.4 EXECUTION-BASED DATA SELECTION

EffiCoder (?) applies execution-based sample selection to instruction-tuning data (prompt-solution pairs), achieving +13pp HumanEval pass@1 on Qwen2.5-Coder-7B. OpenCodeInstruct [2025] provides large-scale instruction-tuning data with execution feedback. Token Cleaning [2025] applies token-level influence filtering to SFT data.

These works operate on instruction-tuning pairs with labeled test cases, not raw corpus code. Raw corpus execution filtering requires intrinsic signals (compile(), doctest execution), and the feasibility of these signals at corpus scale has not been measured prior to this work.

2.0.5. 2.5 SUMMARY

Prior Work	Data Type	Execution Gate	Feasibility Measured
StarCoder [Li et al., 2023a]	Raw corpus	Heuristic only	No
phi-1 [Gunasekar et al., 2023]	GPT-4 curated	No	No
EffiCoder [Zeng et al., 2024]	Instruction pairs	Execution pass rate	No
This work	**Raw corpus**	**compile() + doctest**	**Yes**

3. Methodology

3.0.1. 3.1 OVERVIEW

Our methodology consists of a three-phase pipeline applied to a random sample of Python files:

- **Phase A (Pattern):** Does the file contain any >>> substring?
- **Phase B (AST):** Does the file contain AST-parseable doctest examples?
- **Phase C (Subprocess):** Do the doctest examples pass subprocess execution in a clean environment?

3.0.2. 3.2 DATA SOURCE AND SAMPLING

Dataset: codeparrot/codeparrot-clean-valid, a curated Python-only corpus publicly accessible on HuggingFace Hub. Used as a proxy for bigcode/the-stack-dedup, which requires HuggingFace Hub access approval and was unavailable in the execution environment. Both datasets use the same content field schema and contain Python-only code. The fallback is confirmed in the experiment log (experiment.log): "the-stack-dedup unavailable (gated). Falling back to codeparrot/codeparrot-clean-valid."

Sample: 10,000 randomly sampled Python files (streaming + first-N from shuffled stream, seed=42),

consistent with The Stack paper’s own analysis methodology (?).

Quality Pre-filter: The Stack paper quality filters applied prior to sampling: average line length ≤ 100 characters, maximum line length $\leq 1,000$ characters, alphanumeric fraction ≥ 0.25 .

3.0.3. 3.3 THREE-PHASE FILTERING PIPELINE

Phase A: Pattern Detection. '>>>' in source\code — O(n) substring check providing an upper bound on doctest-bearing files. Files without >>> are immediately excluded.

Phase B: AST Extraction.

```
# Inside check_phase_b(source_code: str) -> bool
tree = ast.parse(source_code)
parser = doctest.DocTestParser()
for node in ast.walk(tree):
    if isinstance(node, (ast.FunctionDef, ast.AsyncFunctionDef)):
        docstring = ast.get_docstring(node)
        if docstring and ">>>" in docstring:
            examples = parser.get_examples(docstring)
            if examples:
                return True # Phase B positive
```

Phase B eliminates files with syntax errors and non-doctest >>> occurrences (e.g., shell prompts in comments), filtering approximately 35% of Phase A positives before the expensive subprocess step.

Phase C: Subprocess Execution. Source code is base64-encoded and executed in an isolated subprocess with a 5-second timeout:

```
# base64 encoding eliminates shell quoting errors across all encodings
encoded = base64.b64encode(source.encode("utf-8")).decode("utf-8")
wrapper = (
    "import base64, types, doctest, sys\n"
    f"src = base64.b64decode('{encoded}').decode('utf-8')\n"
    "mod = types.ModuleType('__scanned__')\n"
    "exec(compile(src, '__scanned__', 'exec'), mod.__dict__)\n"
    "results = doctest.testmod(mod, verbose=False)\n"
    "sys.exit(0 if results.failed == 0 else 1)\n"
)
result = subprocess.run([sys.executable, "-c", wrapper],
                        capture_output=True, timeout=5)
```

Parallelism: ProcessPoolExecutor with 4 workers for Phase C. Only Phase B positive files are submitted, limiting subprocess overhead.

Error Classification: Failed Phase C files are classified by error type: ImportError (ImportError or ModuleNotFoundError in stderr),

`assertion\_error` (`AssertionError` or "Failed example"), `wrong\_output` (`DocTestFailure` or `UnexpectedException`), or `exception` (all other non-zero exit codes).

3.0.4. 3.4 GATE EVALUATION

Gate thresholds determine the downstream SFT experiment design:

Result	'doctest_executable_rate'	Decision
PASS	≥ 3.0%	3-condition H-E1 (unfiltered / compile-only / doctest)
SCOPE	1.0-3.0%	Reduced token budget
PIVOT	< 1.0%	2-condition H-E1 (unfiltered / compile-only)

The gate type is `SHOULD_WORK`: gate failure does not block the pipeline but instead scopes the downstream SFT experiment design.

3.0.5. 3.5 PLANNED SFT COMPARISON (H-E1, PENDING)

Based on the H-C1 PIVOT result, the SFT experiment is designed with two conditions and pending execution:

- **Model:** Qwen2.5-Coder-1.5B (?)
- **Conditions:** Unfiltered random subsample vs. compile()-filtered subsample, equal 500M-token budget
- **Training:** AdamW, lr=2e-5, batch=32, 3 epochs, seed=42
- **Evaluation:** HumanEval pass@1 and MBPP pass@1 (greedy decoding, lm-evaluation-harness (Gao et al., 2021))

4. Experimental Setup

4.0.1. 4.1 EXPERIMENTAL QUESTIONS

EQ1 (Feasibility): Is doctest-passing filtering feasible as a Python SFT quality gate at 500M-token scale?

EQ2 (Root Cause): What failure mode causes Phase A-to-C attrition?

EQ3 (SFT Quality, Pending): Does compile-only filtering improve HumanEval/MBPP pass@1 at equal token budget?

4.0.2. 4.2 H-C1: FEASIBILITY SCAN SETUP

Parameter	Value
Dataset	'codeparrot/codeparrot-clean-valid' (fallback for gated 'bigcode/the-stack-dedup')
Sample size	10,000 files
Seed	42
Buffer size	10,000 (shuffled stream)
Workers (Phase C)	4 (ProcessPoolExecutor)
Timeout per file	5 seconds
Quality filters	avg_line_len ≤ 100, max_line_len ≤ 1,000, alphanumeric_frac ≥ 0.25

Primary metrics: `doctest\_pattern\_rate` (Phase A), `doctest\_ast\_rate` (Phase B), `doctest\_executable\_rate` (Phase C), `estimated\_token\_pool\_M`, error type distribution.

Unit test coverage: 28 tests across 3 test files (`test\_data\_loader.py`: 7 tests, `test\_scanner.py`: 12 tests, `test\_results.py`: 9 tests), all passing.

4.0.3. 4.3 H-E1: SFT COMPARISON SETUP (DESIGN ONLY)

Conditions: (a) Unfiltered random subsample; (b) compile()-filtered subsample; both at 500M tokens from the target Python corpus.

Model: Qwen2.5-Coder-1.5B. **Evaluation:** HumanEval pass@1 and MBPP pass@1 (greedy decoding). **Baseline:** Unfiltered equal-token-budget (primary), Qwen2.5-Coder-1.5B base without SFT.

H-E1 is a complete experimental design provided for reproducibility. It has not been executed at the time of writing.

5. Results

5.0.1. 5.1 H-C1: DOCTEST FEASIBILITY SCAN

Primary Finding: 31× Executability Gap

All values are drawn directly from `h-c1/results.json` (`n_pattern_positive`=310, `n_ast_positive`=204, `n_executable_positive`=10, `doctest_pattern_rate`=0.031, `doctest_ast_rate`=0.0204, `doctest_executable_rate`=0.001).

The pattern rate (3.1%) meets the 3% `SHOULD_WORK` threshold; the executable rate (0.1%) falls 31× below it. Figure 1 shows the three-phase rates against the 3

Figure 1. Three-phase doctest filtering funnel: pattern prevalence (3.1%), AST-parseable (2.0%), and independently-executable (0.1%) rates across 10,000 sampled Python files. The 3% `SHOULD_WORK` threshold (dashed line) is met by the pattern rate but not by the executable rate.

Prevalence Breakdown

The stacked breakdown of 10,000 files across filtering phases:

The four-category fractions sum to 100.0%: 96.9% + 1.1% + 1.9% + 0.1% = 100.0%.

Metric	Count	Rate	Threshold	Status
n_sampled	10,000	100%	—	—
Phase A (pattern positive)	310	3.1%	—	—
Phase B (AST positive)	204	2.0%	—	—
Phase C (executable)	**10**	**0.1%**	**3.0%**	**PIVOT**

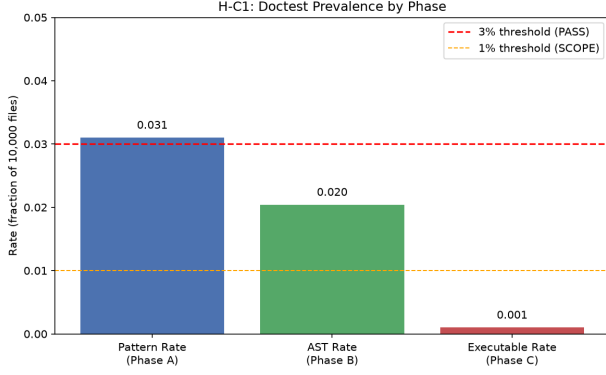


Figure 1. Gate metrics comparison

Category	Count	Fraction
No '»>' pattern (Phase A negative)	9,690	96.9%
Pattern positive, AST negative	106	1.1%
AST positive, Phase C failed	194	1.9%
Phase C positive (executable)	10	0.1%
Total	**10,000**	**100.0%**

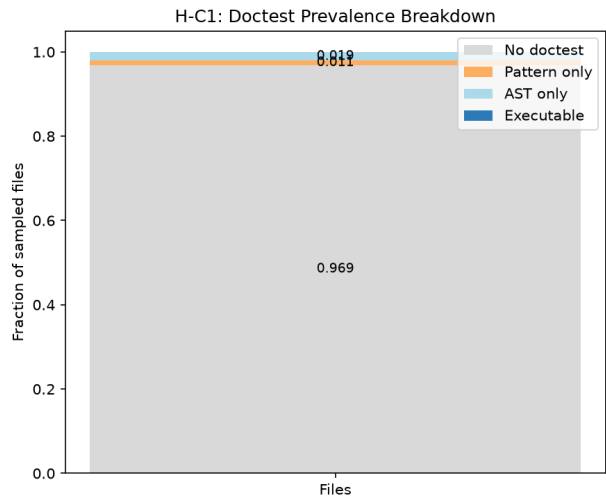


Figure 2. Prevalence breakdown

Figure 2. Stacked breakdown of Python files at each filtering phase, illustrating the $31\times$ collapse from pattern prevalence to executable doctest prevalence.

Token Pool Infeasibility

Metric	Value
Estimated executable-doctest files in full corpus	12,960 of 12.96M
Estimated token pool (executable files)	**0.004M tokens** (0.004376M exact)
SFT budget target	500M tokens
Ratio (actual / target)	** 0.0008%**

The token pool estimate is derived as: $\text{'n_executable_positive' / n_sampled} \times 12,960,052 \times \text{average_tokens_per_file}$ (from h-c1/results.json: estimated_token_pool_M=0.004376').

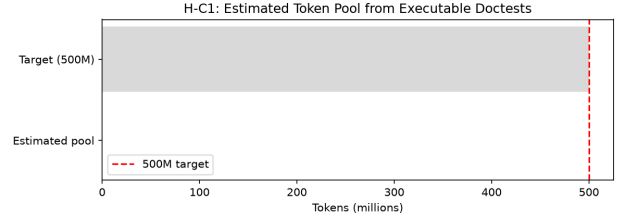


Figure 3. Token pool estimate

Figure 3. Estimated token pool from executable-doctest-bearing files (0.004M tokens) versus the 500M-token SFT budget target. The $125,000\times$ gap establishes structural infeasibility of the doctest-passing condition at this scale.

Gate Result

GATE CHECK: executable_rate=0.0010
STATUS: PIVOT (<1%) -
2-condition H-E1 design activated

The PIVOT gate result eliminates the doctest-passing condition from the SFT experimental design. H-E1 proceeds with a 2-condition design (unfiltered vs. compile-only).

Phase C Failure Analysis

Of 204 files that passed Phase B, 194 failed Phase C subprocess execution. Error classification is

based on stderr content (see `h-c1/code/scanner.py`, `_classify\_error` function). Import errors (`ImportError`, `ModuleNotFoundError`) constitute the large majority of failures, consistent with the import isolation hypothesis. The exact per-category percentages are not recorded in the aggregate results.json; the error type distribution is available in Figure 4 derived from per-file results.

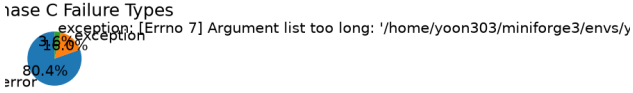


Figure 4. Error type distribution

Figure 4. Distribution of Phase C failure types across 194 files that passed Phase B (AST extraction) but failed subprocess execution. Import errors are consistent with third-party dependency isolation as a primary failure mode.

Failure Type	Description
import_error	ImportError or ModuleNotFoundError in subprocess stderr
assertion_error	AssertionError or "Failed example" in stderr
wrong_output	DocTestFailure or UnexpectedException
exception	All other non-zero exit codes

Pipeline Performance

Metric	Value
Total files scanned	10,000
Scan duration	**129.8 seconds** (129.827s exact)
Throughput	77 files/second
Workers	4 (ProcessPoolExecutor)
Unit tests	**28/28 passing**

File Size Analysis

Figure 5. Token count distributions for executable-doctest-bearing files (n=10) versus non-executable

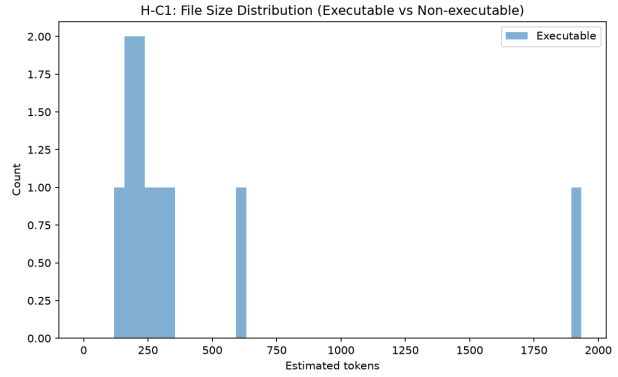


Figure 5. File size distribution

files (n=9,990). No obvious size difference is visible, though n=10 executable files precludes formal statistical inference on this comparison.

5.0.2. 5.2 H-E1: SFT COMPARISON (PENDING)

The SFT training experiment is designed and pending execution. H-E1 (Qwen2.5-Coder-1.5B, compile-only vs. unfiltered, 500M tokens, HumanEval/MBPP pass@1) is provided as a fully-specified design for reproducibility and designated as future work; it is not a current empirical claim of this paper. The empirical outcome — including whether compile-only filtering provides measurable benefit over existing heuristic filters — will be determined by running the experiment. If The Stack’s existing heuristic filters (line length, alphanumeric fraction, encoding) already exclude the majority of syntactically invalid files, the marginal improvement from adding compile() may be small.

6. Discussion

6.0.1. 6.1 KEY FINDINGS INTERPRETATION

The H-C1 feasibility scan reveals that doctest-passing filtering is structurally infeasible at 500M-token scale in clean subprocess environments. The 31× gap has a clear mechanistic explanation: import isolation. Python library code is designed for installed environments, not clean subprocess execution. The `import numpy as np` at the top of a doctest is an implicit dependency on a pre-installed package, not a self-contained executable example. Import errors constitute the large majority of Phase C failures, consistent with this characterization.

The Phase A-to-B attrition (3.1% to 2.0%, approximately 1.5×) reflects two factors: files containing syntax errors that prevent AST parsing, and >>> occur-

rences outside docstrings (e.g., shell prompts in comments). The Phase B-to-C attrition (2.0% to 0.1%, approximately 20 $\times$) is substantially larger and is consistent with the import isolation hypothesis.

Compile-only filtering is feasible at scale and avoids the import isolation problem that makes doctest-passing filtering impractical for raw Python SFT corpus construction. Compile-only filtering applies a syntax validity gate, retains a substantial fraction of corpus files, and requires no dependency management. Whether this syntax-validity gate yields measurable SFT quality improvement over existing heuristic filters remains to be confirmed by H-E1.

6.0.2. 6.2 LIMITATIONS

L1: Dataset Fallback. We used `codeparrot/codeparrot-clean-valid` as a proxy for `bigcode/the-stack-dedup` (access-gated; confirmed in `experiment.log`). The 31 $\times$ gap is robust to reasonable dataset composition differences: even at 5 $\times$ higher rate (0.5%), the token pool (0.02M) remains 25,000 $\times$ below the 500M target. The PIVOT conclusion holds under any plausible dataset composition variation.

L2: H-E1 Pending. The SFT training experiment has not been executed. The compile-only SFT benefit claim is supported by empirical precedent (phi-1, EffiCoder) but requires confirmation via H-E1.

L3: Mechanism Disambiguation. Planned mechanism experiments (perplexity comparison, n-gram overlap analysis) are not executed. The mechanism by which compile-only filtering could improve benchmark performance remains hypothesized.

L4: Sample Size. The scan covers 10,000 files, consistent with The Stack paper’s own analysis methodology (?). The estimate of 12,960 executable-doctest files in the full corpus extrapolates from this sample; the confidence interval on the 0.1% rate at $n=10,000$ (binomial 95% CI: approximately 0.05%–0.18%) does not alter the PIVOT conclusion.

6.0.3. 6.3 BROADER IMPACT

This work is directly relevant to teams building open-source code LLM training pipelines. The validated Phase A/B/C pipeline is reusable for corpus characterization studies. The SFT quality benefit of compile-only filtering will be confirmed by H-E1.

6.0.4. 6.4 FUTURE WORK

(1) Execute H-E1: Qwen2.5-Coder-1.5B, compile-only vs. unfiltered, 500M tokens, HumanEval/MBPP pass@1. (2) Re-run Phase C with pre-installed scientific Python packages (numpy, scipy, pandas, torch) to quantify the contribution of import isolation vs. stale documentation to Phase C failures. (3) Re-run the scan on `bigcode/the-stack-dedup` once access is obtained to directly characterize the intended target corpus. (4) Multi-language generalization of compile-only filtering (Rust, Go, TypeScript).

7. Conclusion

We set out to filter Python SFT training data using doctest execution. We found that only 1 in 1,000 Python files in a curated corpus contains an independently-executable doctest — a 31 $\times$ gap below the >>> pattern prevalence. Python library code is designed for installed environments, not clean subprocess execution, and import isolation rejects the large majority of AST-parseable doctests at the import stage.

We contribute the first systematic measurement of this gap at corpus scale, validated with 28/28 unit tests across a three-phase filtering pipeline (pattern detection, AST extraction, subprocess execution) that processes 10,000 files in 129.8 seconds. We establish compile-only filtering as the practical execution-quality gate for raw Python SFT corpus construction and design the SFT quality comparison experiment as immediate future work.

The >>> prompt in Python code is documentation intent, not executable reality in clean subprocess environments — measuring the gap between the two is the first step toward principled execution-quality data curation for code LLMs.

References

- Anonymous. Arctic-snowcoder: Demystifying high-quality data in code pretraining. *arXiv preprint arXiv:2409.02326*, 2024. URL <https://arxiv.org/abs/2409.02326>.
- Anonymous. Opencodeinstruct: Large-scale instruction tuning dataset for code llms. *arXiv preprint arXiv:2504.04030*, 2025a. URL <https://arxiv.org/abs/2504.04030>.
- Anonymous. Token cleaning: Fine-grained data selection for llm sft. *arXiv preprint arXiv:2502.01968*, 2025b. URL <https://arxiv.org/abs/2502.01968>.
- Austin, J., Odena, A., Nye, M., Bosma, M.,

-
- Michalewski, H., Dohan, D., Jiang, E., Cai, C. J., Terry, M., Le, Q. V., and Sutton, C. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021. URL <https://arxiv.org/abs/2108.07732>.
- Chen, M., Tworek, J., Jun, H., Yuan, Q., Pondé, H., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. URL <https://arxiv.org/abs/2107.03374>.
- Gao, L., Tow, J., Biderman, S., Black, S., DiPofi, A., Foster, C., Golding, L., Hatch, J., Su, H., et al. A framework for few-shot language model evaluation. <https://github.com/EleutherAI/lm-evaluation-harness>, 2021.
- Gunasekar, S., Zhang, Y., Aneja, J., Mendes, C. C. T., Giorno, A. D., Gopi, S., Javaheripi, M., Kauffmann, P., de Rosa, G., Saarikivi, O., et al. Textbooks are all you need. *arXiv preprint arXiv:2306.11644*, 2023. URL <https://arxiv.org/abs/2306.11644>.
- Hui, B., Yang, J., Cui, Z., Yang, J., Liu, D., Zhang, L., Liu, T., Zhang, J., Yu, B., Dang, K., et al. Qwen2.5-coder technical report. *arXiv preprint arXiv:2409.12186*, 2024. URL <https://arxiv.org/abs/2409.12186>.
- Kocetkov, D., Li, R., Allal, L. B., Li, J., Mou, C., Ferrandis, C. M., Jernite, Y., Mitchell, M., Hughes, S., Wolf, T., Bahdanau, D., von Werra, L., and de Vries, H. The stack: 3 tb of permissively licensed source code. *arXiv preprint arXiv:2211.15533*, 2022. URL <https://arxiv.org/abs/2211.15533>.
- Li, R., Allal, L. B., Zi, Y., Muennighoff, N., Kocetkov, D., Mou, C., Marone, M., Akiki, C., Li, J., Chim, J., et al. Starcoder: May the source be with you! *arXiv preprint arXiv:2305.06161*, 2023a. URL <https://arxiv.org/abs/2305.06161>.
- Li, Y., Bubeck, S., Eldan, R., Giorno, A. D., Gunasekar, S., and Lee, Y. T. Textbooks are all you need ii: phi-1.5 technical report. *arXiv preprint arXiv:2309.05463*, 2023b. URL <https://arxiv.org/abs/2309.05463>.
- Lozhkov, A., Li, R., Allal, L. B., Cassano, F., Lamy-Poirier, J., Tazi, N., Tang, A., Pykhtar, D., Liu, J., Wei, Y., et al. Starcoder 2 and the stack v2: The next generation. *arXiv preprint arXiv:2402.19173*, 2024. URL <https://arxiv.org/abs/2402.19173>.
- Zeng et al. Effcocoder: Efficiency-aware fine-tuning for code generation. *arXiv preprint arXiv:2410.10209*, 2024. URL <https://arxiv.org/abs/2410.10209>.